MODERNISATION

The Legacy Modernisation Roadmap

How to assess an ageing system, choose a strategy and move forward without breaking the business.

SyntecHire

E-book

INSIDE THIS E-BOOK

Contents

01 Is it really a problem?

02 Map what you have

03 Choose a strategy for each part

04 Build the safety net

05 Replace gradually

06 Move the data

07 Measure and keep going

What you will learn

- Measure the real cost and risk of the current system
- Choose between rehost, replatform, refactor, rearchitect and rebuild
- Build tests, monitoring and release routines before changing core code
- Replace a system gradually instead of with a risky big-bang rewrite
- Move data with checks at every stage
- Track progress with a handful of outcome measures

Who it is for

- CTOs and heads of engineering responsible for ageing systems
- Operations leaders who rely on software they cannot easily change
- Teams planning a migration or replatforming project

CHAPTER 01

Is it really a problem?

Questions that separate a working old system from a risky one.

Age alone is not a reason to modernise. A stable system that is understood, supported and rarely changed can be left alone. The signs of real risk show up in how the system behaves when you try to change it.

Measure before you decide. Four numbers tell most of the story.

Key points

- Lead time from request to release
- Frequency of incidents after releases
- Number of people who can change the core code safely
- Components out of vendor support or missing patches

CHAPTER 02

Map what you have

Build a shared picture of components, data and integrations.

List the applications, databases, scheduled jobs and every integration, including the unofficial ones such as spreadsheets and file drops. Talk to the people who run the system daily. They know the workarounds that never reached the documentation.

Record what you find in a short document. It prevents surprises and gives new engineers a starting point.

Key points

- Components and owners
- Data stores and their sizes
- Overnight and monthly jobs
- Third-party dependencies

CHAPTER 03

Choose a strategy for each part

Five options, from moving the system to rebuilding it.

Modernisation is a spectrum. Rehosting moves the application as it is. Replatforming makes small changes to use managed services. Refactoring improves the structure of the code. Rearchitecting changes how the system is divided. Rebuilding replaces it.

Treat the system as a set of parts and pick a strategy for each. Keep what is stable, move what is costly to host, improve what changes often and replace what cannot be repaired.

Key points

- Start with a part that is painful and well bounded
- Prefer reversible steps
- Avoid doing everything at once

CHAPTER 04

Build the safety net

Tests, releases, monitoring and rollback before touching the core.

You are about to change working code, so you need to know when something breaks. Put automated tests around the behaviour that matters most, even if they are coarse. Make builds and releases repeatable. Add monitoring for errors, slow requests and failed jobs. Make sure you can roll back quickly.

This work feels like a delay. It is what lets everything after it move faster.

Key points

- Tests that call the system from outside
- One-command build and release
- Alerts that reach a person
- A rehearsed rollback

CHAPTER 05

Replace gradually

The strangler approach, one capability at a time.

Put a routing layer in front of the old system, then move one capability at a time to a new implementation. Requests for the migrated capability go to the new code and everything else continues to the old system.

Each step is small, reversible and delivers value by itself. Over months the old system shrinks until it can be switched off. Compare this with a big-bang rewrite, where nothing ships for a year and the old system keeps changing underneath.

Key points

- Pick the first capability carefully
- Release every few weeks
- Keep the old system until the new one has proved itself

CHAPTER 06

Move the data

Source of truth, replication, checks and a rehearsed cutover.

Data is usually the hardest part. Decide which system is the source of truth at each stage. Keep old and new data stores in step with replication or change capture. Check record counts and key totals before and after every move.

Rehearse the final cutover on a copy of production data, and write the rollback plan before you start.

Key points

- Define the source of truth per stage
- Compare totals, not only counts
- Rehearse, then rehearse again

CHAPTER 07

Measure and keep going

Outcome measures that show whether the programme is working.

Return to the numbers you started with. Track lead time for changes, release frequency, incident rate and the time a new engineer needs to make a first change.

Report progress in business terms: faster releases, fewer incidents, shorter onboarding. Adjust the plan when the numbers do not move, because activity is not the same as progress.

Key points

- Review the measures every month
- Share results with stakeholders
- Fund the next step from what you learned

NEXT STEP

Talk to an engineer

If you would like help applying this guide to your own project, SyntecHire can review your plan and suggest a practical way forward. Request a conversation on our website and a senior engineer will reply within two working days.